

Лабораторная работа № 9

Основы Docker и контейнеризация приложений

Продолжительность

2 занятия по 90 минут.

Среда выполнения

- VirtualBox;
 - Ubuntu Server или Debian Server;
 - одна виртуальная машина docker01;
 - установленный Docker Engine;
 - установленный Docker Compose Plugin;
 - доступ в Интернет для получения образа Nginx.
-

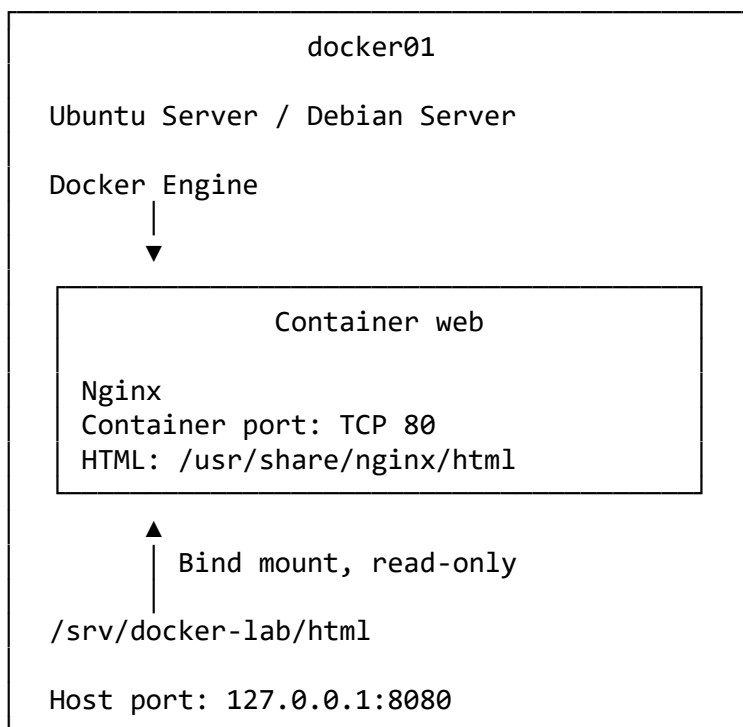
1. Цель работы

Научиться разворачивать, диагностировать и воспроизводимо запускать контейнерное приложение.

В ходе работы необходимо:

- проверить Docker Client и Docker Server;
 - получить образ из Registry;
 - запустить контейнер Nginx;
 - безопасно опубликовать порт только на Loopback хоста;
 - изучить жизненный цикл контейнера;
 - подключить HTML-каталог через read-only bind mount;
 - проверить журналы и состояние контейнера;
 - выполнить диагностические команды через `docker exec`;
 - собрать собственный Docker Image;
 - подготовить `.dockerignore`;
 - описать запуск через `compose.yaml`;
 - настроить Restart Policy, Healthcheck и ротацию журналов;
 - проверить удаление и повторное создание контейнера.
-

2. Схема стенда



Сервис публикуется только на Loopback-адресе:

`127.0.0.1:8080`

Он не должен быть доступен через внешний IP-адрес `docker01`.

3. Рабочие каталоги

В лаборатории используется следующая структура:

```
/srv/docker-lab
├── html
│   └── index.html
├── image
│   ├── Dockerfile
│   ├── .dockerignore
│   └── index.html
└── compose
    ├── compose.yaml
    ├── html
    │   └── index.html
```

4. Правила безопасности

Во время работы необходимо соблюдать ограничения:

- команды Docker выполняются через `sudo`;
- пользователь не добавляется в группу `docker`;
- Docker Socket не подключается внутрь контейнера;
- режим `--privileged` не используется;
- корневой каталог хоста не подключается в контейнер;
- порт публикуется только на `127.0.0.1`;
- HTML-каталог подключается только для чтения;
- секреты не передаются через командную строку;
- команды `docker system prune` и `docker volume prune` не выполняются.

Доступ к:

`/var/run/docker.sock`

фактически позволяет управлять Docker Daemon и может дать широкие административные возможности на хосте.

5. Базовые задания

Задание 1. Проверить Docker Engine

Проверить службу:

```
sudo systemctl status docker --no-pager
```

Ожидаемое состояние:

```
active (running)
```

Если служба не запущена:

```
sudo systemctl enable --now docker
```

Проверить Client и Server:

```
sudo docker version
```

Команда должна показать два раздела:

```
Client
Server
```

Проверить общую информацию:

```
sudo docker info
```

Проверить Compose:

```
sudo docker compose version
```

Проверить Docker Socket:

```
sudo ls -l /var/run/docker.sock
```

Посмотреть существующие объекты:

```
sudo docker ps
sudo docker ps -a
sudo docker image ls
sudo docker volume ls
sudo docker network ls
```

Ожидаемый результат

- Docker Daemon работает;
 - Docker Client подключается к Daemon;
 - Compose Plugin доступен;
 - ошибок соединения с Docker Socket нет.
-

Задание 2. Получить и исследовать образ Nginx

Получить фиксированный учебный образ:

```
sudo docker pull nginx:stable-alpine
```

Проверить список образов:

```
sudo docker image ls
```

Найти:

```
nginx    stable-alpine
```

Посмотреть подробную информацию:

```
sudo docker image inspect nginx:stable-alpine
```

Вывести только несколько параметров:

```
sudo docker image inspect \
  --format 'id={{.Id}} architecture={{.Architecture}} os={{.Os}}' \
  nginx:stable-alpine
```

Посмотреть команду запуска:

```
sudo docker image inspect \
  --format 'entrypoint={{json .Config.Entrypoint}} cmd={{json .Config.Cmd}}' \
  nginx:stable-alpine
```

Посмотреть слои образа:

```
sudo docker image history nginx:stable-alpine
```

Что нужно определить

- Repository;
- Tag;
- Image ID;
- размер образа;
- архитектуру;
- команду запуска.

Вывод

Tag — это удобное имя образа, но его содержимое может изменяться.

Image ID и Digest идентифицируют конкретное содержимое точнее.

Задание 3. Запустить первый контейнер

Запустить Nginx:

```
sudo docker run \  
  --name web \  
  -d \  
  -p 127.0.0.1:8080:80 \  
  nginx:stable-alpine
```

Проверить запущенные контейнеры:

```
sudo docker ps
```

Проверить публикацию порта:

```
sudo docker port web
```

Ожидаемый результат:

```
80/tcp -> 127.0.0.1:8080
```

Проверить порт на Linux-хосте:

```
sudo ss -lntp | grep ':8080'
```

Проверить приложение:

```
curl -I http://127.0.0.1:8080/
```

Ожидаемый ответ:

```
HTTP/1.1 200 OK
```

Получить страницу:

```
curl http://127.0.0.1:8080/
```

Проверить адрес хоста:

```
ip -br addr
```

Попробовать обратиться через внешний IP docker01:

```
curl \
  --connect-timeout 3 \
  http://<IP-ДОКЕР-ХОСТА>:8080/
```

Ожидаемый результат

- через 127.0.0.1:8080 сервис доступен;
- через внешний адрес хоста порт 8080 недоступен;
- контейнер работает в фоновом режиме.

Почему это важно

Команда:

```
-p 8080:80
```

обычно публикует порт на всех интерфейсах хоста.

В лаборатории используется более безопасный вариант:

```
-p 127.0.0.1:8080:80
```

Задание 4. Исследовать жизненный цикл контейнера

Посмотреть только работающие контейнеры:

```
sudo docker ps
```

Остановить:

```
sudo docker stop web
```

Проверить:

```
sudo docker ps
sudo docker ps -a
```

Контейнер должен находиться в состоянии:

Exited

Проверить сайт:

```
curl \
  --connect-timeout 3 \
  http://127.0.0.1:8080/
```

Запустить тот же контейнер:

```
sudo docker start web
```

Проверить:

```
sudo docker ps
curl -I http://127.0.0.1:8080/
```

Перезапустить:

```
sudo docker restart web
```

Посмотреть Container ID:

```
sudo docker inspect \
  --format '{{.Id}}' \
  web
```

Важно

Команда:

```
docker start web
```

запускает уже существующий контейнер.

Команда:

```
docker run ...
```

создаёт новый контейнер из образа.

Повторный `docker run --name web` приведёт к конфликту имени, пока старый контейнер существует.

Задание 5. Подключить HTML через read-only bind mount

Удалить первый контейнер:

```
sudo docker rm -f web
```

Проверить:

```
sudo docker ps -a
```

Создать каталог:

```
sudo mkdir -p /srv/docker-lab/html
```

Создать страницу:

```
sudo tee /srv/docker-lab/html/index.html >/dev/null <<'EOF'
<!doctype html>
<html lang="ru">
<head>
    <meta charset="utf-8">
    <title>Docker Lab</title>
</head>
<body>
    <h1>Docker Lab – bind mount</h1>
    <p>Страница хранится на Linux-хосте.</p>
</body>
</html>
EOF
```

Проверить:

```
sudo ls -l /srv/docker-lab/html
```

Запустить контейнер с bind mount:

```
sudo docker run \
    --name web \
    -d \
    -p 127.0.0.1:8080:80 \
    --mount \
    type=bind,source=/srv/docker-lab/html,target=/usr/share/nginx/html,readonly \
    nginx:stable-alpine
```

Проверить:

```
curl http://127.0.0.1:8080/
```

Посмотреть Mounts:

```
sudo docker inspect \
    --format '{{json .Mounts}}' \
    web
```

Проверить попытку записи из контейнера:

```
sudo docker exec web \
    sh -c 'touch /usr/share/nginx/html/container-file.txt'
```

Ожидается ошибка:

Read-only file system

Изменить страницу на хосте:

```
sudo sed -i \
    's/bind mount/bind mount – изменено на хосте/' \
    /srv/docker-lab/html/index.html
```


Повторить:

```
curl http://127.0.0.1:8080/
```

Вывод

Bind mount связывает каталог контейнера с реальным путём хоста.

Параметр readonly запрещает контейнеру изменять подключённые файлы.

Задание 6. Проверить сохранение данных после пересоздания

Удалить контейнер:

```
sudo docker rm -f web
```

Проверить, что HTML сохранился:

```
sudo cat /srv/docker-lab/html/index.html
```

Повторно создать контейнер:

```
sudo docker run \
  --name web \
  -d \
  -p 127.0.0.1:8080:80 \
  --mount \
  type=bind,source=/srv/docker-lab/html,target=/usr/share/nginx/html,readonly \
  nginx:stable-alpine
```

Проверить:

```
curl http://127.0.0.1:8080/
```

Ожидаемый результат

- контейнер удалён и создан заново;
- данные на хосте не удалены;
- новый контейнер показывает актуальную страницу.

Вывод

Writable Layer удаляется вместе с контейнером.

Bind-mounted каталог живёт отдельно от жизненного цикла контейнера.

Bind mount и Volume сохраняют данные, но сами по себе не являются резервной копией.

Задание 7. Выполнить диагностику контейнера

Проверить состояние:

```
sudo docker ps
sudo docker ps -a
```

Посмотреть последние строки журнала:

```
sudo docker logs \
  --tail 20 \
  web
```

Создать несколько HTTP-запросов:

```
for i in 1 2 3; do
  curl -s http://127.0.0.1:8080/ >/dev/null
done
```

Повторно посмотреть журнал:

```
sudo docker logs \
  --since 2m \
  web
```

Посмотреть процессы:

```
sudo docker top web
```

Посмотреть использование ресурсов:

```
sudo docker stats --no-stream web
```

Проверить пользователя внутри контейнера:

```
sudo docker exec web id
```

Посмотреть файлы:

```
sudo docker exec web \
  ls -la /usr/share/nginx/html
```

Проверить конфигурацию Nginx:

```
sudo docker exec web \
  nginx -t
```

Вывести IP-адрес контейнера:

```
sudo docker inspect \
  --format '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' \
  web
```

Проверить Privileged Mode:

```
sudo docker inspect \
  --format 'privileged={{.HostConfig.Privileged}}' \
  web
```

Ожидается:

```
privileged=false
```

Важно

docker exec используется для диагностики.

Ручное изменение файлов внутри работающего контейнера не является надёжным способом настройки: изменения исчезнут после пересоздания.

Задание 8. Собрать собственный Docker Image

Создать каталог:

```
sudo mkdir -p /srv/docker-lab/image
```

Перейти:

```
cd /srv/docker-lab/image
```

Создать страницу:

```
sudo tee index.html >/dev/null <<'EOF'
<!doctype html>
<html lang="ru">
<head>
  <meta charset="utf-8">
  <title>Custom Image</title>
</head>
<body>
  <h1>lab-nginx:1.0</h1>
  <p>Страница добавлена в образ через Dockerfile.</p>
</body>
</html>
EOF
```

Создать Dockerfile:

```
sudo nano Dockerfile
```

Добавить:

```
FROM nginx:stable-alpine
```

```
WORKDIR /usr/share/nginx/html
```

```
COPY index.html ./index.html
```

EXPOSE 80

```
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
  CMD wget -q --spider http://localhost/ || exit 1
```

Создать .dockerignore:

```
sudo nano .dockerignore
```

Добавить:

```
.git
.gitignore
*.pcap
*.pcapng
*.log
*.bak
*.backup
.env
secrets/
```

Проверить файлы:

```
sudo find . \
  -maxdepth 2 \
  -type f \
  -printf '%p\n'
```

Удалить предыдущий контейнер:

```
sudo docker rm -f web
```

Собрать образ:

```
sudo docker build \
  -t lab-nginx:1.0 \
  .
```

Проверить:

```
sudo docker image ls lab-nginx
```

Посмотреть историю:

```
sudo docker image history lab-nginx:1.0
```

Запустить:

```
sudo docker run \
  --name web \
  -d \
  -p 127.0.0.1:8080:80 \
  lab-nginx:1.0
```

Проверить:

```
curl http://127.0.0.1:8080/
```

Посмотреть Health Status:

```
sudo docker inspect \
  --format '{{.State.Health.Status}}' \
  web
```

Сразу после запуска состояние может быть:

starting

После успешной проверки:

healthy

Вывод

Dockerfile описывает содержимое Image.

EXPOSE 80 документирует порт, но не публикует его на хосте.

Реальная публикация выполняется через -p.

Задание 9. Подготовить Compose-проект

Удалить контейнер:

```
sudo docker rm -f web
```

Создать каталоги:

```
sudo mkdir -p \
  /srv/docker-lab/compose/html
```

Перейти:

```
cd /srv/docker-lab/compose
```

Создать страницу:

```
sudo tee html/index.html >/dev/null <<'EOF'
<!doctype html>
<html lang="ru">
<head>
  <meta charset="utf-8">
  <title>Docker Compose</title>
</head>
<body>
  <h1>Docker Compose</h1>
  <p>Сервис запущен из compose.yaml.</p>
```

```
</body>
</html>
EOF
```

Назначить обычные права чтения:

```
sudo chmod 755 html
sudo chmod 644 html/index.html
```

Создать compose.yaml:

```
sudo nano compose.yaml
```

Добавить:

```
services:
  web:
    image: nginx:stable-alpine
    container_name: web

    ports:
      - "127.0.0.1:8080:80"

    volumes:
      - "./html:/usr/share/nginx/html:ro"

    restart: unless-stopped

    read_only: true

    tmpfs:
      - /var/cache/nginx
      - /var/run

    security_opt:
      - no-new-privileges:true

    healthcheck:
      test:
        - CMD-SHELL
        - wget -q --spider http://localhost/ || exit 1
      interval: 15s
      timeout: 3s
      retries: 3
      start_period: 5s

    logging:
      driver: json-file
      options:
        max-size: "10m"
        max-file: "3"
```

```
mem_limit: 256m
cpus: 0.5
```

Проверить синтаксис и итоговую конфигурацию:

```
sudo docker compose config
```

Запустить проект:

```
sudo docker compose up -d
```

Проверить:

```
sudo docker compose ps
```

Проверить страницу:

```
curl http://127.0.0.1:8080/
```

Проверить Health Status:

```
sudo docker inspect \
  --format '{{.State.Health.Status}}' \
  web
```

Проверить параметры безопасности:

```
sudo docker inspect \
  --format 'read_only={{.HostConfig.ReadonlyRootfs}}
privileged={{.HostConfig.Privileged}}
restart={{.HostConfig.RestartPolicy.Name}}' \
  web
```

Ожидается:

```
read_only=true privileged=false restart=unless-stopped
```

Посмотреть журналы:

```
sudo docker compose logs \
  --tail 30 \
  web
```

Посмотреть созданную сеть:

```
sudo docker network ls
```

Ожидаемый результат

Compose создал:

- контейнер web;
- отдельную сеть проекта;
- публикацию только на 127.0.0.1;
- read-only bind mount;

- Healthcheck;
 - Restart Policy;
 - ограничение журналов;
 - лимиты CPU и RAM.
-

Задание 10. Проверить отказ, Restart Policy и пересоздание

Часть 1. Проверить Restart Policy

Посмотреть текущее число перезапусков:

```
sudo docker inspect \
  --format '{{.RestartCount}}' \
  web
```

Завершить главный процесс Nginx:

```
sudo docker exec web \
  nginx -s quit
```

Подождать несколько секунд:

```
sleep 5
```

Проверить:

```
sudo docker compose ps
```

Посмотреть число перезапусков:

```
sudo docker inspect \
  --format '{{.RestartCount}}' \
  web
```

Проверить приложение:

```
curl -I http://127.0.0.1:8080/
```

Ожидаемый результат

Контейнер автоматически запускается снова, потому что главный процесс завершился, а задана политика:

```
unless-stopped
```

Часть 2. Проверить ручную остановку

Остановить сервис:

```
sudo docker compose stop web
```


Проверить:

```
sudo docker compose ps -a
```

Подождать:

```
sleep 5
```

Контейнер не должен автоматически запуститься после осознанной ручной остановки.

Запустить:

```
sudo docker compose start web
```

Проверить:

```
curl -I http://127.0.0.1:8080/
```

Часть 3. Пересоздать проект

Остановить и удалить контейнер и сеть проекта:

```
sudo docker compose down
```

Проверить сохранность страницы:

```
cat html/index.html
```

Повторно запустить:

```
sudo docker compose up -d
```

Проверить:

```
sudo docker compose ps  
curl http://127.0.0.1:8080/
```

Вывод

`docker compose down` удаляет контейнеры и обычную сеть проекта.

Bind-mounted данные на хосте сохраняются.

Команда:

```
docker compose down -v
```

дополнительно удаляет Named Volumes проекта и может уничтожить хранящиеся в них данные. В базовой части работы она не выполняется.

6. Приёмочная матрица

№	Проверка	Ожидаемый результат	Фактический результат
1	<code>docker version</code>	Видны Client и Server	
2	<code>docker image ls</code>	Есть <code>nginx:stable-alpine</code>	
3	<code>docker ps</code>	Контейнер web работает	
4	<code>docker port web</code>	<code>127.0.0.1:8080 → 80</code>	
5	HTTP через Loopback	Возвращается страница	
6	HTTP через внешний IP	Порт 8080 недоступен	
7	Bind mount	Отображается страница хоста	
8	Запись в bind mount	Запрещена	
9	Удаление контейнера	HTML на хосте сохраняется	
10	<code>docker logs</code>	Видны HTTP-запросы	
11	<code>docker inspect</code>	<code>privileged=false</code>	
12	Custom Image	<code>lab-nginx:1.0</code> запускается	
13	Compose Config	Проверка завершается без ошибок	
14	Healthcheck	Состояние healthy	
15	Restart Policy	Контейнер восстанавливается после сбоя PID 1	
16	<code>compose down</code>	Bind-mounted данные сохраняются	

7. Дополнительные задания

Дополнительное задание 1. Исследовать Named Volume

Создать Volume:

```
sudo docker volume create webdata
```

Проверить:

```
sudo docker volume ls
sudo docker volume inspect webdata
```

Запустить временный контейнер:

```
sudo docker run \
  --name volume-test \
  --rm \
  -v webdata:/data \
```

```
alpine \  
sh -c 'echo "Named Volume Data" > /data/test.txt'
```

Создать второй контейнер и прочитать данные:

```
sudo docker run \  
--rm \  
-v webdata:/data:ro \  
alpine \  
cat /data/test.txt
```

Ожидаемый результат:

Named Volume Data

Удалить контейнеры не требуется: они запускались с --rm.

Проверить, что Volume существует:

```
sudo docker volume ls
```

Удалить Volume только после проверки:

```
sudo docker volume rm webdata
```

Вывод

Named Volume сохраняется независимо от конкретного контейнера.

Удаление самого Volume удаляет находящиеся в нём данные.

Дополнительное задание 2. Проверить пользовательскую Docker Network

Создать сеть:

```
sudo docker network create appnet
```

Запустить Nginx без публикации порта:

```
sudo docker run \  
--name internal-web \  
-d \  
--network appnet \  
nginx:stable-alpine
```

Запустить временный клиент в той же сети:

```
sudo docker run \  
--rm \  
--network appnet \  
curlimages/curl \  
http://internal-web/
```

Контейнер-клиент должен найти сервер по имени:

```
internal-web
```

Проверить сеть:

```
sudo docker network inspect appnet
```

Удалить контейнер:

```
sudo docker rm -f internal-web
```

Удалить сеть:

```
sudo docker network rm appnet
```

Вывод

В пользовательской Docker Network контейнеры получают встроенное DNS-разрешение по имени.

Для внутреннего общения публикация порта на хосте не требуется.

Дополнительное задание 3. Смоделировать Unhealthy Container

Перейти в Compose-проект:

```
cd /srv/docker-lab/compose
```

Открыть:

```
sudo nano compose.yaml
```

Временно изменить Healthcheck:

```
healthcheck:
  test:
    - CMD-SHELL
    - wget -q --spider http://localhost/missing-page || exit 1
  interval: 5s
  timeout: 3s
  retries: 2
  start_period: 2s
```

Проверить:

```
sudo docker compose config
```

Пересоздать:

```
sudo docker compose up -d --force-recreate
```

Наблюдать:

```
watch -n 2 \  
  sudo docker inspect \  
  --format '{{.State.Status}} {{.State.Health.Status}}' \  
  web
```

Для выхода:

Ctrl+C

Ожидается:

running unhealthy

Проверить HTTP:

```
curl -I http://127.0.0.1:8080/
```

Главная страница при этом может продолжать работать.

Вывод

Healthcheck сообщает о логическом состоянии приложения.

Docker Engine не всегда перезапускает контейнер только из-за статуса unhealthy.

После проверки вернуть правильный URL:

```
http://localhost/
```

И пересоздать сервис:

```
sudo docker compose up -d --force-recreate
```

8. Основные команды Docker

Информация

```
sudo docker version  
sudo docker info  
sudo docker compose version
```

Images

```
sudo docker pull <image>  
sudo docker image ls  
sudo docker image inspect <image>  
sudo docker image history <image>
```

Containers

```
sudo docker run  
sudo docker ps  
sudo docker ps -a
```

```
sudo docker stop <container>
sudo docker start <container>
sudo docker restart <container>
sudo docker rm <container>
```

Диагностика

```
sudo docker logs <container>
sudo docker exec <container> <command>
sudo docker top <container>
sudo docker stats --no-stream <container>
sudo docker inspect <container>
```

Хранилище и сети

```
sudo docker volume ls
sudo docker volume inspect <volume>
sudo docker network ls
sudo docker network inspect <network>
```

Build

```
sudo docker build -t <image:tag> .
```

Compose

```
sudo docker compose config
sudo docker compose up -d
sudo docker compose ps
sudo docker compose logs
sudo docker compose stop
sudo docker compose start
sudo docker compose down
```

9. Требования к отчёту

В отчёте представить:

1. Название и цель работы.
2. Схему контейнерного приложения.
3. Версии Docker Client, Server и Compose.
4. Результат `docker image ls`.
5. Результат `docker ps`.
6. Публикацию порта контейнера.
7. Результат проверки через `curl`.
8. Конфигурацию `read-only bind mount`.
9. Пример журналов контейнера.
10. Результат проверки `privileged`.

11. Dockerfile.
12. .dockerignore.
13. Результат сборки lab-nginx:1.0.
14. Итоговый compose.yaml.
15. Результат Healthcheck.
16. Заполненную приёмочную матрицу.
17. Краткий вывод по работе.

Не требуется добавлять полный вывод `docker inspect`. Достаточно привести параметры, подтверждающие конфигурацию.

В отчёт нельзя включать:

- содержимое Docker Socket;
 - пароли и токены Registry;
 - секретные Environment Variables;
 - файлы .env с реальными данными;
 - приватные ключи;
 - содержимое каталога secrets.
-

10. Чек-лист выполнения

- ☐ Docker Daemon находится в состоянии active.
- ☐ `docker version` показывает Client и Server.
- ☐ Получен образ `nginx:stable-alpine`.
- ☐ Контейнер `web` запущен.
- ☐ Порт опубликован только на `127.0.0.1`.
- ☐ Nginx отвечает через `curl`.
- ☐ Проверены `stop`, `start` и `restart`.
- ☐ HTML подключён через `bind mount`.
- ☐ `Bind mount` работает в режиме `read-only`.
- ☐ Данные сохраняются после удаления контейнера.
- ☐ Проверены `logs`, `exec`, `top` и `stats`.
- ☐ Контейнер не использует `Privileged Mode`.
- ☐ Создан Dockerfile.
- ☐ Создан .dockerignore.
- ☐ Собран образ `lab-nginx:1.0`.
- ☐ Создан `compose.yaml`.
- ☐ Compose Config проходит проверку.
- ☐ Настроена ротация журналов.

- ☐ Настроен Healthcheck.
 - ☐ Настроена Restart Policy.
 - ☐ Настроен no-new-privileges.
 - ☐ Установлены лимиты CPU и RAM.
 - ☐ Проверен отказ главного процесса.
 - ☐ Проверено повторное создание контейнера.
 - ☐ После завершения временные объекты удалены.
-

11. Контрольные вопросы

1. Чем контейнер отличается от виртуальной машины?
2. Чем Docker Image отличается от Container?
3. Чем `docker run` отличается от `docker start`?
4. Почему `-p 127.0.0.1:8080:80` безопаснее, чем `-p 8080:80`?
5. Чем Bind Mount отличается от Named Volume?
6. Почему Volume не является резервной копией?
7. Для чего используются Dockerfile и `compose.yaml`?
8. Почему подключение `/var/run/docker.sock` внутрь контейнера представляет опасность?